Chapter 4

API app setup, env files, database connection

Quan Nguyen[1]

October 2026

Table of Contents

1. Setting Up the FastAPI Application

2. Configuring the PostgreSQL Database

Table of Contents

1 Setting Up the FastAPI Application

2 Configuring the PostgreSQL Database

The Big Picture: What Are We Building Today?

Up to now, the chatbot app has been a single script — it runs, it responds, and everything it "remembers" disappears the moment it stops. Today we turn it into a **real, persistent web service**.

Piece	**Question it answers**
API app setup	How is the app organized?
`.env` files	Where do secrets live?
DB connection	Where is data stored?

Today's goal

By the end of this session, a message sent to `/chat` should be permanently saved in PostgreSQL and retrievable later — even after the server restarts.

Why Structure Matters Before We Write Any Code

Imagine handing someone a single 2,000-line `main.py` file and asking them to "find where messages get saved." That's what happens without structure.

- Splitting code into folders is not about "best practice for its own sake" — it's about **answering questions fast**: "where do I add a new endpoint?", "where does the DB code live?"
- Every FastAPI project we'll build from now on follows the **same three-part shape**: `routers` (the doors into the app), `schemas` (the rules for what data looks like), and `db` (how we talk to storage).

From Script to Application: Project Structure

A single `main.py` works for a demo, but grows unmanageable fast — a real FastAPI project separates concerns into folders.

```
app/
 |-- main.py              (creates FastAPI(), includes routers)
 |-- routers/
 |     +-- chat.py         (/chat endpoint lives here)
 |-- schemas/
 |     +-- chat.py          (Pydantic request/response models)
 |-- db/
 |     +-- database.py     (connection pool, session dependency)
 +-- .env                  (secrets, never committed)
```

Introducing Uvicorn: What Is It?

FastAPI describes **what** should happen for each request — but it cannot listen on a network port by itself. Something has to actually accept connections from the internet and hand them to your code. That "something" is **Uvicorn**.

- Uvicorn is an **ASGI server** — a lightweight program that speaks raw HTTP (and WebSockets) and translates it into Python function calls.
- Without a server like Uvicorn, `app = FastAPI()` is just an object sitting in memory — nobody outside your script can reach it.

Why "ASGI" Specifically?

You may have heard of **WSGI** (used by older frameworks like Flask/Django). FastAPI instead uses **ASGI** — the difference matters for a chat app.

	WSGI	**ASGI**
Style	Synchronous only	Synchronous **and** async
Handles	One request at a time per worker	Many concurrent requests
Good for	Simple request/response	Chat, streaming, WebSocket

Why this matters for our chatbot

A chat endpoint often waits on a slow database query or an LLM API call. With ASGI, Uvicorn can serve **other users** while one request is waiting — with WSGI, that request would block the whole worker.

Running Your App with Uvicorn

```
uvicorn app.main:app --reload --port 8000
```

Part	Meaning
`app.main`	Python module path (`app/main.py`)
`:app`	the `FastAPI()` object inside that file
`--reload`	auto-restart server on code changes (dev only)
`--port 8000`	which network port to listen on

Common mistake

Forgetting `--reload` during development means you must manually stop (`Ctrl+C`) and rerun Uvicorn after every code change — easy to forget why "my fix isn't showing up."

What Actually Happens When You Run `uvicorn`?

Before looking at code, it helps to picture the runtime flow in plain words.

1. Python imports `app/main.py` and finds the `app` object (an instance of `FastAPI`).
2. `uvicorn` starts a web server and hands every incoming HTTP request to that `app` object.
3. `app` looks at the request's path and method (e.g., `POST /chat`) and finds the matching function.
4. That function runs, and whatever it `returns` gets converted to JSON and sent back to the caller.

Key idea

`FastAPI()` itself does nothing except hold a map from "path + method" to "Python function." Everything else — validation, docs, routing — is built on top of that simple idea.

Grouping Endpoints with APIRouter

```
# app/routers/chat.py
from fastapi import APIRouter, Depends

router = APIRouter(prefix="/chat", tags=["chat"])

@router.post("/")
async def send_message(...):
    ...

@router.get("/history")
async def get_history(...):
    ...
```

Real case

A larger app might have `users.py`, `chat.py`, and `admin.py` routers — each file owns one feature, and `main.py` stays a short file that just wires

The Problem Pydantic Solves

Before Pydantic, imagine this endpoint:

```
@router.post("/")
async def send_message(msg: dict):
    return msg["content"].upper()
```

- What if the client sends `{"text": "hi"}` instead of `{"content": "hi"}`? The server crashes with a confusing `KeyError` — at 2 AM, in production, with no warning.
- What if `content` is a number instead of a string? `.upper()` crashes too.

Pydantic's job

Describe **once** what a valid request looks like. FastAPI then checks every incoming request against that description **before** your function ever runs — bad requests never reach your code at all.

Validating Data with Pydantic Models

```
# app/schemas/chat.py
from pydantic import BaseModel
from datetime import datetime
class ChatRequest(BaseModel):
    role: str
    content: str
class ChatResponse(BaseModel):
    id: int
    role: str
    content: str
    created_at: datetime
```

- `ChatRequest` validates **incoming** data — a request missing `content` is rejected automatically with `422 Unprocessable Entity`.
- `ChatResponse` shapes **outgoing** data and can hide internal fields that shouldn't reach the client (e.g. passwords).

Using Pydantic Models in the Endpoint

```
from fastapi import APIRouter, Depends
from app.schemas.chat import ChatRequest, ChatResponse
from app.db.database import get_conn
router = APIRouter(prefix="/chat", tags=["chat"])
@router.post("/", response_model=ChatResponse)
async def send_message(
    msg: ChatRequest,
    conn=Depends(get_conn),
):
    row = await conn.fetchrow(
        """INSERT INTO messages (role, content)
           VALUES ($1, $2)
           RETURNING id, role, content, created_at""",
        msg.role, msg.content,
    )
    return dict(row)
```

Dependency Injection, Explained Without Code First

`Depends()` sounds abstract, so start with an analogy: a restaurant kitchen doesn't ask each chef to walk to the farm and pick vegetables — a supplier **delivers** exactly what's needed, when it's needed.

- Every endpoint that touches the database needs a **connection**. Instead of writing "get a connection" code inside every single endpoint, we write it **once** as a function, and FastAPI "delivers" the result to any endpoint that asks for it.
- This is exactly what `Depends(get_conn)` means: "before running this endpoint, call `get_conn` and hand me what it produces."

The next slide shows what `Depends()` does technically — keep the delivery analogy in mind while reading it.

Dependency Injection: What `Depends()` Actually Does

- Used for the database connection (`get_conn`) so every endpoint gets a fresh connection from the pool without repeating that logic.
- Also common for **authentication** (e.g., `get_current_user`) and **shared validation** logic across multiple endpoints.
- FastAPI **caches** a dependency's result within a single request — calling the same dependency twice in one request reuses the first result instead of running it twice.

Why not just call the function directly

Direct calls make endpoints hard to test — dependency injection lets tests **override** `get_conn` with a fake in-memory connection, without touching endpoint code at all.

Table of Contents

1 Setting Up the FastAPI Application

2 Configuring the PostgreSQL Database

Why Applications Need a Database

A running process keeps its data **only in RAM** — the moment the process stops, everything is gone.

- **Volatile state:** restarting a FastAPI server wipes any Python list/dict used as "storage."
- **Durability requirement:** data must survive crashes, redeploys, and server reboots.
- **Concurrency requirement:** many requests need to read and write the same data safely at the same time — something an in-memory Python object cannot guarantee under concurrent load.

Real case

A chatbot API that stores conversation history in a Python list loses *all* chat history on every `uvicorn --reload`. Moving that state into PostgreSQL fixes this permanently.

PostgreSQL: What and Why

PostgreSQL is a free, open-source **relational database management system (RDBMS)** that stores data in tables of rows and columns.

- Enforces a fixed **schema** — every row in a table has the same columns and types, catching bad data early.
- Supports **ACID transactions** — a chain of writes either fully succeeds or fully rolls back, never leaves partial data.
- Widely used in production (Instagram, Reddit, Spotify) — not a toy database.

Terminology mapping

Database → **Table** → **Row (record)** → **Column (field)**. A table `messages` might have one row per chat turn, with columns `id`, `role`, `content`, `created_at`.

Connection String and DATABASE_URL

```
postgresql://user:password@localhost:5432/chatdb
```

Segment	Meaning
`postgresql`	database driver/scheme
`user:password`	login credentials
`localhost:5432`	host and port (5432 = PG default)
`chatdb`	target database name

The Connection String as an Address

Before decoding the syntax, think of a connection string as a **full mailing address** for a specific mailbox.

Address concept	**Connection string piece**
Which city/building	`localhost:5432` (host + port)
Which mailbox inside it	`chatdb` (database name)
Who is allowed to open it	`user:password`

Just like a real address, if any single piece is wrong — wrong city, wrong mailbox number, wrong key — you get denied access, not a "close enough" partial match. This is why a single typo in `DATABASE_URL` causes a hard failure, not a warning.

Why Secrets and Code Must Never Mix

If a password is typed directly into a `.py` file, it becomes part of the **permanent history** of the project the moment it's committed to Git.

- Deleting the line later does **not** remove it — old commits still contain it, and anyone can browse Git history to find it.
- If the repository is ever made public, or a teammate leaves the team, that password is compromised.
- The fix is separation: code describes **what** to do; `.env` holds **which** credentials to use. Only `.env` changes between a laptop, a teammate's machine, and a production server — the code never does.

Never Hardcode Secrets: Loading from `.env`

`.env` file (never committed):

```
DATABASE_URL=postgresql://user:secret@localhost:5432/chatdb
```

Loading it in Python (`python-dotenv`):

```
from dotenv import load_dotenv
import os

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")
if DATABASE_URL is None:
    raise RuntimeError("DATABASE_URL is not set")
```

Typed Settings with Pydantic BaseSettings

Reading raw `os.getenv()` calls scattered across files is error-prone — `pydantic-settings` centralizes and validates every environment variable in one place.

```
# app/core/config.py
from pydantic_settings import BaseSettings
class Settings(BaseSettings):
    database_url: str
    app_env: str = "development"

    class Config:
        env_file = ".env"
settings = Settings()
```

- If `DATABASE_URL` is missing, the app fails with a clear error
- Every other file imports `settings` instead of calling `os.getenv` repeatedly.

From Chat Messages to Table Columns

Before the `CREATE TABLE` syntax, think about what a single chat turn actually **contains**: who said it, what they said, and when.

Real-world fact	**Column**	**Type**
A unique number for this row	`id`	integer
Who sent it (user or bot)	`role`	short text
What was said	`content`	long text
When it happened	`created_at`	timestamp

Designing a table is really just this exercise: list the facts you need to remember, then pick a column and type for each one.

Designing the messages Table

```
CREATE TABLE messages (
    id SERIAL PRIMARY KEY,
    role VARCHAR(20) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT now()
);
```

- `id SERIAL PRIMARY KEY` — auto-incrementing integer, uniquely identifies each row. `SERIAL` is simplest for a single-server app
- `role` — `"user"` or `"assistant"`, distinguishes who sent the message.
- `content` — the actual text; `TEXT` has no length limit, unlike `VARCHAR(n)`.
- `created_at` — defaults to `now()` so the app doesn't need to compute a timestamp manually.

CRUD: The Four Core Operations

Create, **R**ead, **U**pdate, **D**elete — every persistent application boils down to these four operations.

```
-- Create
INSERT INTO messages (role, content)
VALUES ('user', 'Xin chao');

-- Read
SELECT * FROM messages ORDER BY created_at;

-- Update
UPDATE messages SET content = 'Da sua' WHERE id = 1;

-- Delete
DELETE FROM messages WHERE id = 1;
```

In this project

Why One Table Is Rarely Enough

A single `messages` table works for one continuous chat, but real apps support **many separate conversations** per user.

- Without a way to group messages, `SELECT * FROM messages` returns every conversation ever had, mixed together.
- The fix: a second table, `conversations`, and a **link** from each message back to the conversation it belongs to — this link is called a **foreign key**.
- `JOIN` is simply the SQL operation that "follows" that link to pull matching rows from both tables at once.

SQL JOIN: Combining Related Tables

```
CREATE TABLE conversations (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100)
);

ALTER TABLE messages
    ADD COLUMN conversation_id INTEGER
    REFERENCES conversations(id);
```

Fetching all messages with their conversation title:

```
SELECT c.title, m.role, m.content
FROM messages m
JOIN conversations c ON m.conversation_id = c.id
WHERE c.id = 5
ORDER BY m.created_at;
```

Why the App Never Talks to Postgres "Directly"

It may seem simpler to write raw SQL wherever it's needed — so why add a "connection layer" in between?

- Opening a network connection to Postgres takes real time (milliseconds add up under load). Doing this on **every** request, for every user, is wasteful.
- A **pool** solves this by keeping a handful of connections already open and handing them out on demand
- The pool is created **once** when the server starts, and destroyed **once** when it shuts down — not per request.

The Connection Layer: FastAPI Never Touches SQL Directly

```
# database.py
import asyncpg, os

pool: asyncpg.Pool | None = None

async def init_pool():
    global pool
    pool = await asyncpg.create_pool(
        dsn=os.getenv("DATABASE_URL"),
        min_size=2, max_size=10,
    )

async def close_pool():
    if pool:
        await pool.close()
```

Reading Session Logs: Three Common SQL Errors

When something breaks, the terminal/log output tells you exactly where to look — reading it correctly saves hours of guessing.

1. Wrong connection string

`connection refused` — the app cannot reach the database process at all. Check host, port, or whether the DB container is running (`docker ps`).

2. Table does not exist

`relation "messages" does not exist` — table was never created, or the migration was never run against this database.

3. Wrong data type

`invalid input syntax for type integer` — e.g., trying to insert the string `"abc"` into an `INTEGER` column.

The Problem With Typing SQL by Hand Every Time

Running `CREATE TABLE` manually in a local terminal feels fine — until the schema needs to exist somewhere else too.

- A teammate clones the repo — do they know exactly which SQL commands to run, and in what order?
- A production server is deployed — does it get the same schema, with zero typos, every single time?
- A schema changes six months from now — is there any record of what changed, or does someone have to compare tables by eye?

Migrations answer all three questions

A migration file is a **recorded, ordered, repeatable** version of "what SQL to run" — checked into Git just like any other code.

Schema Migrations: Why Not Just Run `CREATE TABLE` by Hand

- **Migration:** a version-controlled, ordered script that changes the database schema (create table, add column, etc.).
- **Alembic** (paired with SQLAlchemy) auto-generates and applies these scripts:

```
alembic revision --autogenerate -m "create messages table"
alembic upgrade head
```

Putting It All Together: The Full Request Journey

Before the checkpoint, trace one `/chat` request through every piece covered today, start to finish.

1. Client sends a POST request with JSON `{"role": "user", "content": "Xin chao"}`.
2. FastAPI's router matches `POST /chat` to the `send_message` function.
3. The body is validated against `ChatRequest` — bad data is rejected here, before anything else runs.
4. `Depends(get_conn)` hands the function a live connection from the pool (built at startup from `DATABASE_URL` in `.env`).
5. An `INSERT` runs against the `messages` table created by an Alembic migration.
6. The inserted row is returned, validated against `ChatResponse`, and sent back as JSON.